Step 2 — Every Run Saved to a Database and Findable by ID — Hints

> Back to this step · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've really tried. Each hint starts with the plain idea, then shows the shape of the real code.

- Hint 1 — What to save, and when
- Hint 2 — Tying log lines to one request, and testing the rows

> Back to this step · Hint 1 · Hint 2 · Solution

Hint 1 — What to save, and when

Every run becomes one row in a SQLite table `runs`: `id, input, output, log, status, created_at` — Doc12's table.

The order matters:

1. **Before** the team runs: insert a row with status `"started"`, and keep its id (`cursor.lastrowid`).
2. After it finishes: update that row — output, log, `"completed"`.
3. If it crashes: update that row to `"failed"`, then return the 500.

That way a crashed run still leaves an honest record. Then add `GET /runs/{run_id}`: select the row, 404 if it's missing, else return `dict(row)`.

Doc12's Build Task split, unchanged:

- `db/database.py` → `DB_PATH`, `get_connection()` (with `row_factory = sqlite3.Row`), `insert_started_run()`, `finish_run()`, `fail_run()`, `fetch_run()` — each write in its **own** `with conn:`.
- `db/models.py` → `Run(BaseModel)`; `output` and `log` are `str | None` — they're empty while a run is `"started"` or `"failed"`.
- `schemas.py` → `TaskResponse` gains `run_id` and `status`.
- `routes.py` → `conn = get_connection()` once, at import time; the `log` column is `"\n".join(outcome["steps"])`.

Always `?` placeholders — never an f-string in SQL.

> Back to this step · Hint 1 · Hint 2 · Solution

Hint 2 — Tying log lines to one request, and testing the rows

Two requests at once write log lines that mix together. Give each request a short ID, made **once** in a middleware, and put it at the start of every log line for that request: `[9ee100c1] run 3 started`. Send it back to the client in an `X-Request-ID` header, too.

Then test that the database really behaves: a valid run is saved, a bad body saves nothing, a crash is saved as `"failed"`, and a wrong ID is a 404.

```
@app.middleware("http")
async def add_request_id(request: Request, call_next):
    request_id = str(uuid.uuid4())[:8]
    request.state.request_id = request_id
    response = await call_next(request)
    response.headers["X-Request-ID"] = request_id
    return response
```

The route reads it with `http_request.state.request_id` — add `http_request: Request` as a second parameter.

In the tests, count rows straight from `routes.conn`: `SELECT COUNT(*) FROM runs` before and after a 422, and `... WHERE status = ?` with `("failed",)` before and after a crash. Read a saved run back with `client.get("/runs/" + str(run_id))`.

Full solution: Show me the solution